DENİZLİ TEKNİK BİLİMLER MESLEK YÜKSEKOKULU

PROJE TEKNOLOJİ KİTAPÇIĞI

Yüz Tanımalı Sınav Giriş Sistemi

Projede kullanılan programlama dilleri, kütüphaneler, backend, frontend, veritabanı ve donanım: ne oldukları, projede nerede ve neden kullanıldıkları, sunumda sorulabilecek sorular.

Python 3.14 · Flask · face_recognition · dlib · OpenCV · NumPy · SQLite · HTML · CSS · JavaScript · Piper TTS · ONNX · openpyxl · pytest · Bash · Linux · ESP32-CAM

Pamukkale Üniversitesi · Denizli Teknik Bilimler MYO · Kasım 2026

İçindekiler

1	Proje bir bakışta	Ne yapar, kim kullanır
2	Mimari: parçalar nasıl birbirine bağlı	Servisler, portlar, veri akışı
3	Teknoloji haritası	Tek tabloda tüm teknolojiler
4	Programlama dili: Python	Neden Python, kullanılan dil özellikleri
5	Yapay zekâ çekirdeği: yüz tanıma	face_recognition, dlib, 128 boyutlu yüz kodu
6	Görüntü işleme: OpenCV, NumPy, Pillow	Kamera, MJPEG, çizim
7	Canlılık kontrolü ve baş yönü	Göz kırpma (EAR), rehberli kayıt
8	Backend: Flask web sunucusu	Rotalar, API, oturum, güvenlik
9	Veritabanı: SQLite ve dosyalar	Tablolar, olay türleri, pickle
10	Frontend: HTML, CSS, JavaScript	Kiosklar, görevli paneli, tarayıcı API'leri
11	Ses: Piper TTS ve ONNX	İnternetsiz Türkçe konuşma
12	Excel çıktısı: openpyxl	Yoklama raporu
13	Donanım ve işletim sistemi	ESP32-CAM, telefon kamerası, Linux, Bash
14	Test: pytest	Otomatik testler
15	Güvenlik ve KVKK	Şifre özeti, çerez, kişisel veri
16	Bir öğrencinin girişi: baştan sona	Adım adım senaryo
17	Sunumda sorulabilecek sorular	Kısa, hazır cevaplar
18	Sözlük	Terimler

1 Proje bir bakışta

Sınav Giriş Sistemi, sınav salonunun kapısındaki bir kameradan gelen görüntüde öğrencinin yüzünü tanıyan, gözünü kırpmasını isteyerek gerçek bir insan olduğunu doğrulayan, ona sesli olarak "Hoş geldiniz" diyip rastgele bir sıra numarası veren ve yoklamayı otomatik tutan bir sistemdir. Görevli, bir web paneli üzerinden öğrencileri kaydeder, sınav oluşturur, yoklamayı canlı izler ve Excel'e aktarır.

Öğrencinin gördüğü

- **Kayıt kiosku** (`/kayit`): numara + ad soyad + KVKK onayı, sonra sesli yönlendirmeyle düz / sağ / sol fotoğraflar kendiliğinden çekilir.
- **Giriş kiosku** (`/kiosk`): kapıdaki tam ekran. Tanınınca "Lütfen gözlerinizi kırpın", sonra "Hoş geldiniz, Sıra No 12".

Görevlinin gördüğü

- **Tanıma**: işaretli canlı görüntü + bugünkü girişler.
- **Öğrenci Kaydı**: elle fotoğraf, düzenleme, silme.
- **Sınavlar**: sınav oluştur, giriş saatleri, canlı yoklama, elle giriş, Excel.

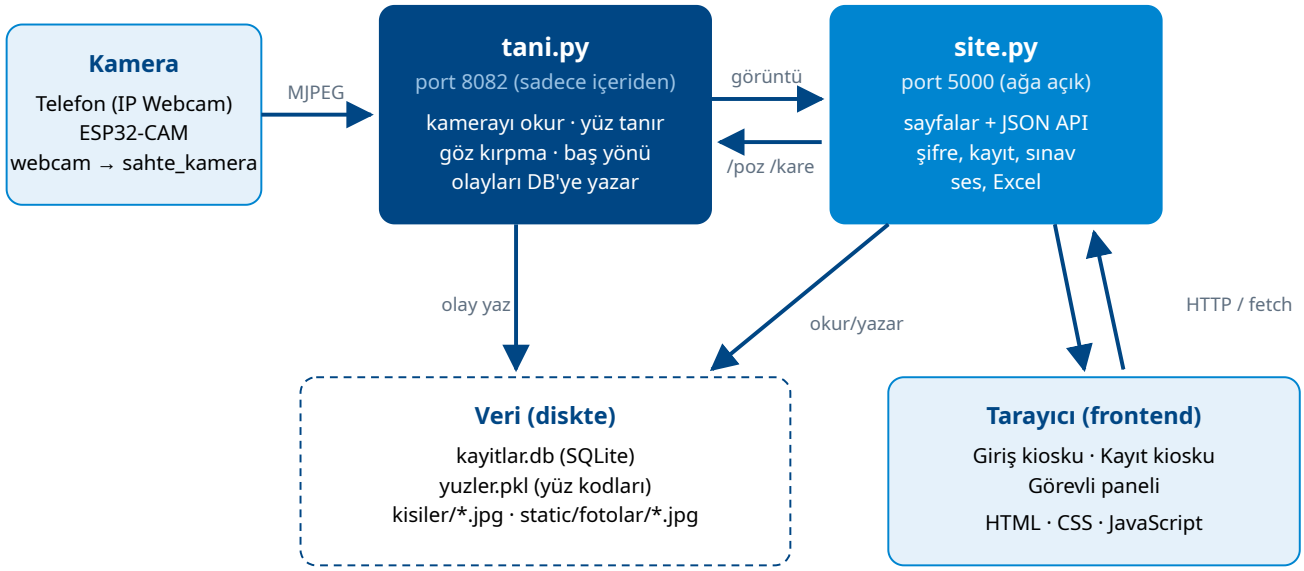
Öne çıkan özellikler

Özellik	Nasıl yapılıyor
Yüz tanıma	Her yüz 128 sayılık bir "yüz koduna" çevrilir, kayıtlı kodlarla mesafe karşılaştırılır (`face_recognition` / `dlib`).
Yanlış eşleşmeye karşı	Aynı kişi art arda 3 analizde (~1 sn) görülmeden kayıt düşülmez; iki kişiye birden benziyorsa "?" der ve karar vermez.
Fotoğrafla kandırmaya karşı	Göz kırpma testi: her karede göz açıklık oranı ölçülür (canlılık kontrolü).
Rehberli kayıt	Burun ile göz ortasının konumundan baş yönü hesaplanır; doğru açıda fotoğraf kendiliğinden çekilir.
Sesli karşılama	Piper ile internetsiz Türkçe konuşma; yoksa tarayıcının sesi.
Sınav kuralları	Giriş penceresi (ör. 14:30–15:00), sadece listedekiler, tek aktif sınav, otomatik başla/bitir, rastgele sıra.
Raporlama	Canlı yoklama sayfası, `.xlsx` dışa aktarma.
Tek tıkla başlatma	Masaüstü simgesi → Bash betiği → tüm servisler + tarayıcı.
Güvenilirlik	124 otomatik test; telefon kamerası yoksa webcam'e otomatik geçiş.

> **Sunumda tek cümleyle**
>
> "Python ile yazılmış, Flask tabanlı bir web uygulaması; derin öğrenme modeliyle yüzü tanıyor, göz kırpmayla canlılığı doğruluyor, SQLite'ta yoklama tutuyor ve tamamen internetsiz çalışıyor."

2 Mimari: parçalar nasıl birbirine bağlı

Sistem, aynı bilgisayarda çalışan **üç ayrı Python programından** (servis) oluşur. Bunları `baslat.py` tek komutla sırayla açar, biri çökerse hepsini kapatır (yarım çalışan sistem kalmasın diye).



Şekil 1 — Üç servis, ortak veri ve tarayıcı. Oklar veri akışının yönünü gösterir.

Dosya	Port	Görevi
`baslat.py`	—	Orkestra şefi: venv'e geçer, kameraya ulaşılıyor mu bakar (ulaşılamazsa webcam'e geçer), servisleri alt süreç (`subprocess`) olarak açar, çıktılarını toplar, Ctrl+C / SIGTERM gelince hepsini düzgünce kapatır.
`sahte_kamera.py`	8081	ESP32-CAM gelene kadar bilgisayarın webcam'ini ESP32 ile aynı formatta (640×480 MJPEG) yayınlar.
`tani.py`	8082	Kameraya bağlanan **tek** program. Yüzleri bulur ve tanır, göz kırpmayı ölçer, kayıt sırasında baş yönünü hesaplar, olayları veritabanına yazar, işaretli görüntüyü yayınlar.
`site.py`	5000	Web sitesi: tüm sayfalar (HTML/CSS/JS gömülü), JSON API, şifre/oturum, öğrenci kaydı, sınav yönetimi, ses, Excel.
`veri.py`	—	Ortak kütüphane: veritabanı şeması, sınav zamanı hesapları, boş sıra seçimi, benzerlik yüzdesi, yüz dosyası okuma/yazma.
`ayarlar.py`	—	Tüm ayarlar tek yerde (kamera adresi, TOLERANS, MARJ, koltuk sayısı...).
`ses.py`	—	Metni Türkçe sese çevirir (Piper), önbellekler.
`kaydet.py` , `hassasiyet.py`	—	Yardımcı araçlar: yüz veritabanını fotoğraflardan baştan kurmak; sınıfa göre tanıma eşiğini ölçmek.

Neden kameraya sadece tani.py bağlanıyor?

ESP32-CAM aynı anda yalnızca **tek izleyiciye** görüntü verebilir. Bu yüzden görüntüyü tek bir program alır, diğerleri (site, tarayıcı) görüntüyü ondan "ikinci elden" alır. Bu aynı zamanda ağır yüz tanıma işini web sunucusundan ayırır: site yavaşlamaz.

Mikroservis benzeri yapı

Her servis kendi işini yapar ve HTTP ile konuşur. Bu, "monolitik" (tek parça) yapıya göre hataları ayırmayı ve parçaları ayrı ayrı test etmeyi kolaylaştırır. Servisler aynı veritabanı dosyasını paylaşır.

3 Teknoloji haritası

Projedeki her teknolojinin hangi katmanda, hangi sürümle ve ne için kullanıldığı (sürümler projenin sanal ortamından alınmıştır):

Teknoloji	Sürüm	Katman	Projede ne işe yarıyor
Python	3.14	Dil	Tüm backend, yüz tanıma, testler.
Flask	3.1	Backend	Web sunucusu: sayfalar, JSON API, görüntü akışı (hem site.py hem tani.py).
Werkzeug	3.1	Backend	Flask'ın altyapısı; ayrıca **şifre özeti** (hash) üretme/kontrol.
Jinja2	3.1	Backend → Frontend	HTML şablonlarına veri yerleştirme (`render_template_string`).
face_recognition	1.3	Yapay zekâ	Yüz bulma, 128 boyutlu yüz kodu, yüz noktaları (göz, burun).
dlib	20.0	Yapay zekâ	face_recognition'ın içindeki C++ makine öğrenmesi kütüphanesi (HOG + ResNet).
OpenCV (cv2)	5.0	Görüntü işleme	Kameradan kare okuma, renk dönüşümü, döndürme, aynalama, JPEG kodlama.
NumPy	2.5	Bilimsel hesap	Görüntüler ve yüz kodları dizi olarak; mesafe, ortalama, yüzdelik hesapları.
Pillow (PIL)	13.0	Görüntü işleme	Görüntünün üstüne **Türkçe karakterli** isim yazmak (OpenCV ç, ş, ğ yazamaz).
SQLite (sqlite3)	yerleşik	Veritabanı	Öğrenciler, olaylar, sınavlar, sınav listeleri.
pickle	yerleşik	Veri	Yüz kodlarını `yuzler.pkl` dosyasına kaydetme.
Piper TTS	1.8	Ses	İnternetsiz Türkçe metinden-sese (sinir ağı modeli).
ONNX Runtime	1.31	Yapay zekâ	Piper'ın ses modelini (`.onnx`) çalıştıran motor.
openpyxl	3.1	Raporlama	Yoklamayı biçimli Excel (`.xlsx`) dosyası olarak üretme.
HTML5 / CSS3	—	Frontend	Tüm ekranlar; CSS Grid, değişkenler, medya sorguları.
JavaScript (ES2020+)	—	Frontend	Canlı güncelleme (fetch), kiosk akışı, ses, tam ekran. Kütüphanesiz ("vanilla").
pytest	9.2	Test	124 otomatik test.
Node.js	24	Test	Sayfalardaki JavaScript kodunun söz dizimi kontrolü (`node --check`).
Bash	—	İşletim sistemi	`sinav_giris.sh` : tek tıkla başlat/durdur.
Linux (Fedora/Nobara)	—	İşletim sistemi	.desktop simgesi, `notify-send` , `xdg-open` , `firewalld` .
venv / pip	—	Ortam	Projeye özel, yalıtılmış Python paket ortamı.

ESP32-CAM, IP Webcam	—	Donanım	Ağ üzerinden MJPEG görüntü veren kamera.

> **requirements.txt**
>
> Projenin bağımlılık listesi: `setuptools<81`, `face_recognition`, `opencv-python`, `flask`, `numpy`, `piper-tts`, `openpyxl`, `pytest`. `pip install -r requirements.txt` ile hepsi kurulur. `setuptools<81` sınırı, face_recognition'ın eski `pkg_resources` modülüne ihtiyaç duymasındandır. dlib bir C++ kütüphanesi olduğu için derlenmesi gerekir; bu yüzden kurulumda önce `cmake gcc-c++ python3-devel` yüklenir.

4 Programlama dili: Python

Projenin backend tarafının tamamı (yaklaşık 3.500 satır uygulama + 1.200 satır test) **Python** ile yazılmıştır. Python yorumlanan (interpreted), dinamik tipli, okunması kolay genel amaçlı bir dildir.

Neden Python?

- **Yapay zekâ ekosistemi:** face_recognition, OpenCV, NumPy, ONNX gibi kütüphanelerin en olgun hali Python'dadır.
- **Hızlı geliştirme:** az kodla çok iş; prototipten çalışan ürüne hızlı geçiş.
- **Ağır işler aslında C/C++'ta:** Python "yapıştırıcı" dildir; yüz tanıma (dlib), görüntü işleme (OpenCV) ve matris hesabı (NumPy) arka planda derlenmiş, hızlı C/C++ koduyla çalışır.
- **Web de aynı dilde:** Flask ile web sunucusu da Python'da, tek dil yeterli.

Projede kullanılan Python özellikleri

Özellik	Projede örnek
Çoklu iş parçacığı (`threading`)	tani.py'de üç iş parçacığı aynı anda çalışır: kamera okuma, yüz tanıma, göz kırpma ölçümü. Ortak değişkenler `threading.Lock` (kilit) ile korunur.
Alt süreçler (`subprocess`)	baslat.py üç servisi ayrı süreç olarak açar ve izler.
Üreteçler (generator, `yield`)	Canlı video akışı: her kare `yield` ile tarayıcıya sırayla gönderilir, bellekte biriktirilmez.
Dekoratörler (`@app.route`)	Bir fonksiyonu bir web adresine bağlar.
Bağlam yöneticileri (`with`)	`with closing(db_ac()) as db:` — veritabanı işi bitince bağlantı kesin kapanır.
Sınıflar	`AkisDurumu`: tanıma döngüsünün kareler arası hafızası.
Sözlük/küme üreteçleri	`{numara: ardisik.get(numara, 0) + 1 for _, numara in bulunanlar}`
Walrus operatörü (`:=`)	`while parca := kaynak.read1(65536): yield parca`
f-string	`f"{ad} ({numara}) için {sayi}. fotoğraf kaydedildi."`
Düzenli ifadeler (`re`)	Ad soyad ve öğrenci numarası doğrulama.
Sinyaller (`signal`)	SIGTERM gelince tüm servisler düzgünce kapanır.
Ortam değişkenleri	baslat.py, seçtiği kamerayı tani.py'ye `SINAV_KAMERA` değişkeniyle bildirir.

Sanal ortam (venv)

`venv/` klasörü projeye özel bir Python kurulumudur; buradaki paketler sistemin geri kalanını etkilemez. baslat.py, venv dışından çalıştırılırsa kendini `os.execv` ile venv'deki Python'la yeniden başlatır — kullanıcının "venv'i etkinleştir" adımını unutması sorun olmaz.

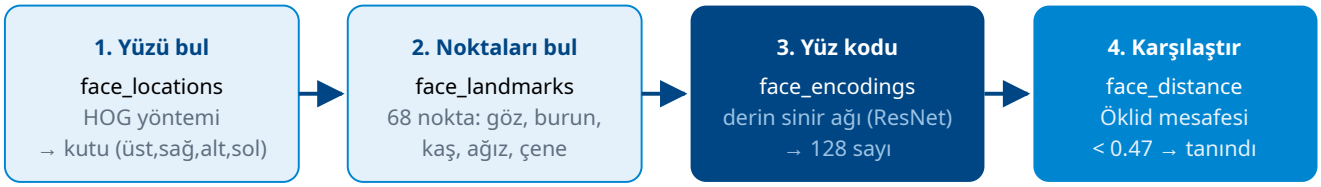
```
# baslat.py — venv dışından çalıştırıldıysa venv'deki Python ile yeniden başla
if sys.prefix == sys.base_prefix and os.path.exists(VENV_PYTHON):
    os.execv(VENV_PYTHON, [VENV_PYTHON, __file__] + sys.argv[1:])
```

> **Kodun dili Türkçe**
>
> Değişken ve fonksiyon adları Türkçedir (`olay_yaz` , `kisi_bul` , `bos_sira_sec`). Bu, kodun ekip ve hocalar tarafından kolay okunmasını sağlar. Python, adlarda ASCII dışı harflere izin verse de projede ç/ş/ğ yerine c/s/g kullanılmıştır (her klavyede yazılabilsin diye).

5 Yapay zekâ çekirdeği: yüz tanıma

Projenin kalbi **face_recognition** kütüphanesidir. Bu kütüphane, C++ ile yazılmış **dlib** makine öğrenmesi kütüphanesinin Python'dan kolay kullanılan bir sarmalayıcısıdır. Önceden eğitilmiş modelleri hazır gelir; projede model **eğitilmez**, sadece kullanılır.

Yüz tanımanın üç adımı



Şekil 2 — Bir kameradaki yüzden "kim olduğu" kararına giden yol.

1. Yüz bulma (detection) — HOG

HOG (Histogram of Oriented Gradients, yönlü gradyan histogramı): görüntüdeki parlaklık değişimlerinin yönlerini küçük hücrelerde sayar ve yüzün tipik "kenar deseni"ne benzeyen bölgeleri bulur. GPU gerektirmez, işlemcide hızlı çalışır. Sonuç, her yüz için bir kutudur: `(üst, sağ, alt, sol)`.

2. Yüz noktaları (landmarks)

Yüzde 68 önemli nokta bulunur (her göz 6 nokta, burun ucu, kaşlar, dudaklar, çene). Proje bunları iki yerde kullanır: **göz kırpma** (göz noktaları) ve **baş yönü** (gözler + burun). Bkz. Bölüm 7.

3. Yüz kodu (face encoding / embedding)

Yüz, önceden milyonlarca yüzle eğitilmiş bir **derin sinir ağına** (ResNet mimarisi, 29 katman) verilir; ağ yüzü **128 sayıdan oluşan bir vektöre** çevirir. Ağ öyle eğitilmiştir ki aynı kişinin farklı fotoğrafları birbirine yakın, farklı kişilerin vektörleri birbirinden uzak düşer. Bu yaklaşıma **metrik öğrenme** (triplet loss) denir. dlib modelinin standart veri setinde (LFW) doğruluğu %99,38'dir.

4. Karşılaştırma — Öklid mesafesi

İki yüz kodu arasındaki mesafe: $\sqrt{(a_1-b_1)^2 + (a_2-b_2)^2 + \dots + (a_{128}-b_{128})^2}$. 0 = aynı fotoğraf; büyüdükçe farklı. Projedeki ölçümde aynı kişi 0,26–0,59, farklı kişiler 0,51–0,85 çıkmıştır.

Projenin karar kuralı (`kisi_bul`)

```
# tani.py — sadeleştirilmiş
mesafeler = face_recognition.face_distance(kodlar, kod)   # tüm kayıtlı fotoğraflara mesafe
# her öğrencinin en yakın fotoğrafını al, yakından uzağa sırala
d1, n1 = siralama[0]
if d1 >= TOLERANS:                    return None       # kimseye benzemiyor → "Bilinmeyen"
if siralama[1][0] - d1 < MARJ:       return BELIRSIZ   # iki kişiye birden benziyor → "?"
return n1                                              # tanındı
```

Ayar	Değer	Anlamı
`TOLERANS`	0,47	En yakın kişi bundan yakın olmalı. Kütüphanenin varsayılanı 0,6'dır; proje daha katıdır (yanlış kişiye giriş vermemek, "tanımamak"tan daha kötüdür).
`MARJ`	0,05	En yakın kişi, ikinciden en az bu kadar daha yakın olmalı. Birbirine benzeyen iki öğrenci (ikizler, kardeşler) karıştırılmaz.
`ONAY_KARE`	3	Aynı kişi art arda 3 analizde görülmeli (tek karelik hatalı eşleşmeler elenir).
`AYRILMA`	3 sn	Kişi bu kadar görünmezse "ayrıldı" sayılır; aynı mesaj durmadan tekrarlanmaz.

Benzerlik yüzdesi

Mesafe herkes için anlaşılır olmadığından ekranda yüzde gösterilir (`veri.benzerlik`). Eşik (0,47) tam **%50**'ye denk gelecek şekilde doğrusal olmayan bir dönüşüm uygulanır: 0,30 → %94, 0,43 → %82, 0,66 → %32. Bu bir olasılık değil, okunabilirlik için bir ölçektir.

Neden her öğrenciden 3 fotoğraf (düz, sağ, sol)?

Bir kişinin her fotoğrafı ayrı bir yüz kodu olarak saklanır; karşılaştırmada **en yakını** kullanılır. Farklı açılar, kapıda başını hafif çevirmiş öğrencinin de tanınmasını sağlar. `hassasiyet.py` her fotoğrafı diğerleriyle tanımaya çalışarak (leave-one-out) sınıfa özel eşiğin güvenli olup olmadığını ölçer.

Yeni öğrenci eklenince yeniden eğitmek gerekiyor mu?

Hayır. Sinir ağı genel "yüz → kod" dönüşümünü zaten öğrenmiştir. Yeni öğrenci için sadece fotoğrafının kodu hesaplanıp listeye eklenir. tani.py, `yuzler.pkl` dosyasının değişme zamanını izler ve yeni yüzleri yeniden başlatmadan yükler.

6 Görüntü işleme: OpenCV, NumPy, Pillow

OpenCV (`cv2`)

Açık kaynak, C++ ile yazılmış, en yaygın bilgisayarlı görü kütüphanesidir. Projede:

Fonksiyon	Ne için
`cv2.VideoCapture(adres)`	Ağ kamerasından (HTTP MJPEG) ya da webcam'den kare okuma. Bağlantı koparsa 2 sn sonra yeniden bağlanır.
`cv2.cvtColor(kare, COLOR_BGR2RGB)`	OpenCV renkleri **BGR** sırasıyla tutar, face_recognition **RGB** ister; dönüştürme şart.
`cv2.rotate`	Telefon dik tutulursa gelen görüntüyü düzeltmek (`KAMERA_DONDUR`).
`cv2.flip(kare, 1)`	Ekranda ayna görüntüsü (sağ elini kaldıran sağda görür). Tanıma aynalanmamış görüntüyle yapılır.
`cv2.imencode(".jpg")`	Kareyi JPEG'e sıkıştırıp tarayıcıya yollamak.
`cv2.imdecode` , `cv2.resize`	Yüklenen fotoğrafı açmak; çok büyükse 1600 piksele küçültmek (yüz bulma hızlanır).
`cv2.imwrite`	Kayıt fotoğrafı ve giriş anı fotoğrafı (yüz çevresinden kırpılmış) kaydetmek.

MJPEG: canlı görüntü nasıl taşınıyor?

MJPEG (Motion JPEG), videoyu art arda gönderilen JPEG resimler olarak taşır. HTTP'de `multipart/x-mixed-replace` içerik türüyle tek bir bağlantı açık kalır ve her yeni resim bir öncekinin yerine geçer. Tarayıcı bunu sıradan bir `<img>` etiketinde video gibi gösterir; ek oynatıcı gerekmez. ESP32-CAM ve IP Webcam de bu formatı kullanır.

```
# tani.py — her kare bir "parça"; tarayıcı bir öncekinin yerine koyar
def jpeg_parcasi(kare):
    _, jpg = cv2.imencode(".jpg", kare)
    return b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + jpg.tobytes() + b"\r\n"

Response(kareler(), mimetype="multipart/x-mixed-replace; boundary=frame")
```

NumPy

Python'un bilimsel hesap kütüphanesi. OpenCV'de her görüntü bir NumPy dizisidir (480 × 640 × 3 sayı). Yüz kodları da 128 elemanlı NumPy vektörleridir. Projede: `np.linalg.norm` (iki nokta arası uzaklık — göz açıklığı, göz arası mesafe), `np.mean` (göz/burun merkezi), `np.percentile` (kişinin "normal" göz açıklığı), `np.argmin` (en yakın yüz), `np.frombuffer` (yüklenen dosyayı diziye çevirme). NumPy döngüleri C'de çalıştırdığı için saf Python'dan onlarca kat hızlıdır.

Pillow (PIL)

OpenCV'nin yazı fonksiyonu (`putText`) sadece ASCII karakterleri çizebilir; "Yağız Güneş" yazılamaz. Bu yüzden isim etiketleri Pillow ile, sistemdeki DejaVu/Noto yazı tipi kullanılarak çizilir: görüntü Pillow'a çevrilir, kutu ve yazı çizilir, tekrar OpenCV formatına döndürülür.

Performans ayarları

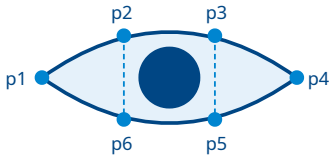
- Yüz analizi en fazla saniyede ~5 kez yapılır (`ISLEM_ARALIGI = 0.2`) — işlemci yorulmaz, görüntü akışı akıcı kalır.
- Kamera okuma ve tanıma ayrı iş parçacıklarındadır: tanıma yavaş kalsa bile görüntü donmaz; tanıma her zaman **en son** kareyi işler (eski kareler kuyrukta birikmez).
- Göz kırpma ölçümü yüzü yeniden aramaz, bilinen kutuyu kullanır: kare başına ~1,5 ms.

7 Canlılık kontrolü ve baş yönü

İki özellik de yüz noktalarını (landmarks) basit geometriyle kullanır; ek bir yapay zekâ modeli gerekmez.

Göz kırpma — Göz Açıklık Oranı (EAR)

Bir yüz fotoğrafı ya da telefon ekranındaki resim göz kırpamaz. Bu yüzden tanınan öğrenciden içeri alınmadan önce gözlerini kırpması istenir. Her gözün 6 noktası vardır (p1…p6). **EAR** (Eye Aspect Ratio), Soukupová ve Čech'in (2016) önerdiği yöntemdir:



$$EAR = (|p2-p6| + |p3-p5|) / (2 \cdot |p1-p4|)$$

Açık göz ≈ 0,25 · Kapalı göz ≈ 0,08

Yani "gözün yüksekliği / genişliği"

Şekil 3 — Göz açıklık oranı: göz kapanınca dikey mesafeler küçülür, oran düşer.

Projedeki algoritma (`goz_olcumu_ekle`):

1. Ayrı bir iş parçacığı (`canlilik_izle`) kameradan gelen **her** karede iki gözün ortalama EAR'ını ölçer (son 90 ölçüm saklanır).
2. Kişinin "normal açık göz" değeri, ölçümlerin **90. yüzdeliği** alınarak bulunur — herkesin gözü farklı açık olduğu için sabit eşik yerine kişiye uyarlanır.
3. Göz bu değerin **%72**'sinin altına inerse "kapandı"; ardından %85'in üstüne çıkarsa "kırptı" denir.
4. 7 saniye içinde kırpılmazsa olay `canlilik_yok` olarak yazılır ve kioskta "Canlılık doğrulanamadı" görünür.
5. Giriş fotoğrafı gözler **açıkken** çekilen kareden alınır.

> **Bilinen sınır (dürüstçe söyleyin)**
>
> Göz kırpan birinin **videosu** oynatılırsa bu kontrol aşılabilir. Daha güçlü yöntemler: rastgele hareket isteme ("sola bakın, gülümseyin"), derinlik/kızılötesi kamera, doku analizi yapan sahtecilik tespit modelleri. Bunlar gelecek çalışma olarak sunulabilir.

Baş yönü — rehberli kayıt (`bas_yonu`)

Kayıt kioskunda öğrenciden sırayla düz, sağa ve sola bakması istenir; sistem doğru açıyı ~0,5 sn koruyunca fotoğrafı kendisi çeker. Yön şöyle hesaplanır:

```
oran = (burun_ucu.x − iki_göz_ortası.x) / göz_arası_mesafe
|oran| < 0,08  → düz bakıyor      oran ≤ −0,20 → sağa     oran ≥ 0,20 → sola
```

Düz bakarken burun iki gözün tam ortasındadır; baş dönünce burun bir göze yaklaşır. Göz arası mesafeye bölmek, kişinin kameraya uzak/yakın olmasından bağımsız bir oran verir. Hızlı olması için burada dlib'in 5 noktalı küçük modeli (`model="small"`) kullanılır. Kamera aynalı olsa da çalışması için ilk yan pozda hangi yöne dönüldüyse kabul edilir, sonra diğer yan beklenir.

Kayıt modu

Kayıt sayfası açıkken tani.py "kayıt modu"na geçer ve giriş olayı yazmaz; yoksa formu dolduran (henüz kaydı bitmemiş) kişiye kiosk "Kaydınız yok" derdi. Sayfa her saniye `/api/poz` 'u çağırarak modu 3'er saniye uzatır; sayfa kapanınca mod kendiliğinden biter. Bu tekniğe "kalp atışı" (heartbeat) denir.

8 Backend: Flask web sunucusu

Flask, Python'un hafif ("mikro") web çatısıdır (framework). Django'ya göre çok daha az hazır parça içerir; bu küçük ve tek bilgisayarda çalışan proje için sade ve yeterlidir. Flask iki yerde kullanılır: `site.py` (asıl web sitesi) ve `tani.py` (iç görüntü/durum servisi).

Temel kavramlar

Kavram	Açıklama ve projeden örnek
Rota (route)	Bir URL'yi bir Python fonksiyonuna bağlar: `@app.route("/kiosk")` .
HTTP metotları	`GET` veri okur (sayfa, liste), `POST` değişiklik yapar (öğrenci ekle, sınav başlat).
JSON API	`/api/...` ile başlayan adresler HTML değil JSON döndürür (`jsonify`); sayfadaki JavaScript bunları okur.
Şablon (Jinja2)	HTML içine `{{ değişken }}` , `{% if %}` ile veri yerleştirme. Projede şablonlar ayrı dosyada değil, site.py içinde metin olarak durur (`render_template_string`) — tek dosya, kolay taşınır.
Ara katman (`before_request`)	Her istekten önce çalışır: oturum açılmamışsa sayfaları `/giris` 'e yönlendirir, API'ye 401 döner.
Oturum (session)	Giriş yapıldığını tarayıcıdaki imzalı çerezde tutar; 30 gün geçerli.
Akış yanıtı (streaming)	`Response(üreteç)` ile canlı video sonsuz akış olarak gönderilir.
Dosya gönderme	`send_file` ile Excel, `send_from_directory` ile öğrenci fotoğrafları.
threaded=True	Her istek ayrı iş parçacığında: bir tarayıcı video izlerken diğer istekler beklemez.

Önemli adresler (endpoint)

Adres	Ne yapar
`GET /giris` , `POST /giris`	Şifre belirleme (ilk açılış, sadece bu bilgisayardan) ve giriş.
`/tanima` , `/kisiler` , `/sinavlar`	Görevli paneli sayfaları.
`/kiosk` , `/kayit`	Giriş kiosku, kayıt kiosku.
`/canli`	tani.py'deki canlı görüntüyü tarayıcıya aktarır (vekil / proxy).
`GET /api/olaylar?son=ID`	Kioskun 400 ms'de bir sorduğu: son ID'den sonraki yeni olaylar + aktif sınav + canlılık durumu.
`GET /api/ses/<id>`	Olayın Türkçe sesini WAV olarak verir.
`POST /api/kisi_ekle`	Fotoğraf al → tam 1 yüz var mı → bu yüz başka numarada kayıtlı mı → kaydet.
`POST /api/sinav_olustur`	Sınav ve öğrenci listesi oluşturur (kurallar kontrol edilir).
`POST /api/sinav/<id>/durum`	Başlat / Bitir (aynı anda tek aktif sınav).

`POST /api/sinav/<id>/elle_giris`	Görevli onayıyla giriş.
`GET /sinav/<id>/excel`	Yoklamayı .xlsx olarak indirir.

tani.py'nin iç API'si (sadece 127.0.0.1)

`/canli` (işaretli görüntü), `/ham` (işaretsiz), `/kare` (son kare, `?yon=sag` ile o yöne bakarken yakalanan kare), `/poz` (baş yönü + kayıt modu), `/canlilik` (göz kırpma bekleme durumu), `/izleme?test=1` (test modu). Bu servis yalnızca `127.0.0.1`'i dinler; ağdaki başka bir cihaz doğrudan erişemez, her şey site.py üzerinden (şifre arkasından) geçer.

Neden WebSocket değil de "yoklama" (polling)?

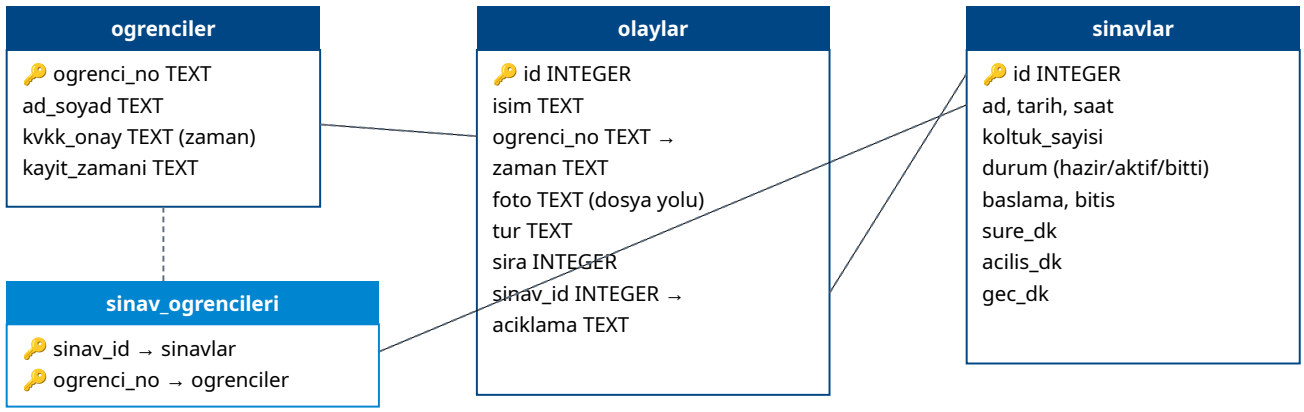
Kiosk, sunucuya 400 ms'de bir "yeni olay var mı?" diye sorar. WebSocket daha anlık olurdu ama ek kütüphane ve karmaşıklık getirirdi. Tek bir kiosk ve küçük JSON yanıtları için yoklama basit, sağlam ve yeterince hızlıdır (insan algısında 0,4 sn gecikme fark edilmez). Bağlantı koparsa bir sonraki denemede kendiliğinden düzelir.

9 Veritabanı: SQLite ve dosyalar

SQLite, sunucu gerektirmeyen, tek bir dosyada (`kayitlar.db`) çalışan ilişkisel (SQL) veritabanıdır. Python'un içinde hazır gelir (`import sqlite3`), kurulum istemez. Telefonlarda, tarayıcılarda, uçaklarda kullanılan, dünyada en yaygın veritabanıdır.

> **Neden MySQL/PostgreSQL değil?**
> Sistem tek bilgisayarda, bir sınav salonu ölçeğinde (yüzlerce öğrenci, binlerce olay) çalışır. Ayrı bir veritabanı sunucusu kurmak, şifrelemek, yönetmek gereksiz yük olurdu. SQLite bu ölçekte hızlı, güvenilir ve yedeklemesi kolaydır (dosyayı kopyalamak yeter). İleride çok salonlu merkezi bir sisteme geçilirse PostgreSQL'e taşımak kolaydır çünkü standart SQL kullanılmıştır.

Tablolar



Şekil 4 — Veritabanı şeması (🔑 birincil anahtar). sinav_ogrencileri, öğrenciler ve sınavlar arasındaki çoka-çok ilişkiyi kurar.

Olay türleri (`olaylar.tur`)

tur	Kioskta	Ne zaman
`giris`	✓ Hoş geldiniz, Sıra No 12	İlk giriş; boş sıralardan rastgele biri verilir.
`tekrar`	! Zaten giriş yaptınız (09:12)	Aynı sınavda / bugün daha önce girmiş.
`listede_yok`	! Bu sınava kaydınız yok	Kayıtlı ama aktif sınavın listesinde değil.
`bilinmeyen`	✕ Kaydınız yok	Sistemde kaydı olmayan yüz.
`canlilik_yok`	✕ Canlılık doğrulanamadı	7 sn içinde göz kırpmadı.
`erken` / `gec`	Giriş henüz açılmadı / kapandı	Giriş penceresi dışında geldi.

Projede kullanılan veritabanı teknikleri

- **Parametreli sorgular:** `db.execute("... WHERE ogrenci_no = ?", (numara,))`. Kullanıcı girdisi SQL metnine eklenmez → **SQL enjeksiyonu** saldırısı engellenir.

- **Şema göçü (migration):** `db_ac()` her açılışta tabloyu `CREATE TABLE IF NOT EXISTS` ile oluşturur, `PRAGMA table_info` ile eksik kolonları bulup `ALTER TABLE ... ADD COLUMN` ile ekler. Proje geliştikçe eski veritabanı silinmeden güncellenir.
- **Birleşik birincil anahtar:** `PRIMARY KEY (sinav_id, ogrenci_no)` — bir öğrenci bir sınava iki kez eklenemez.
- **Eşzamanlılık:** tani.py ve site.py aynı dosyaya yazar; `timeout=10` ile kilitli anlarda bekler.
- **Toplu sorgular:** `COALESCE(MAX(id), 0)`, `LIKE '2026-11-11%'` (bugünün olayları), `ORDER BY ... LIMIT`.

Veritabanı dışındaki veriler

Dosya	İçerik
`yuzler.pkl`	(numaralar listesi, 128'lik yüz kodları listesi). Python'un **pickle** biçimi: NumPy dizilerini olduğu gibi hızlıca kaydedip okur. **Atomik yazma:** önce `.tmp` dosyasına yazılır, sonra `os.replace` ile tek hamlede yer değiştirilir — tani.py asla yarım dosya okumaz.
`kisiler/<no>_<sıra>.jpg`	Kayıt fotoğrafları.
`static/fotolar/`	Her olayın anındaki yüz fotoğrafı (milisaniyeli zaman damgasıyla adlandırılır).
`sifre.txt`, `gizli_anahtar.txt`	Şifre özeti ve çerez imza anahtarı; dosya izni `600` (sadece sahibi okuyabilir).
`sesler/onbellek/*.wav`	Üretilmiş ses cümleleri (adı cümlenin SHA-1 özeti).

> **Sınav zaman mantığı (veri.py)**
> Giriş penceresi = [başlangıç − `acilis_dk`, başlangıç + `gec_dk`). Örnek: 15:00 sınavı, 30/0 → giriş 14:30–15:00. `otomatik_sinav_durumu` her 5 sn'de çalışır: giriş açılınca sınavı `aktif`, süre dolunca `bitti` yapar. Sıra numarası `bos_sira_sec` ile verilmemiş sıralardan `random.choice` ile seçilir (kopya riskine karşı rastgele oturma).

10 Frontend: HTML, CSS, JavaScript

Frontend, hiçbir dış kütüphane (React, Bootstrap, jQuery...) ya da internetten yazı tipi kullanmadan, saf **HTML5 + CSS3 + JavaScript** ("vanilla JS") ile yazılmıştır. Sebep: sistem sınav salonunda **internet olmadan** da aynı görünmeli ve çalışmalıdır. Sayfalar site.py içinde metin olarak durur ve Jinja2 ile sunulur.

Ekranlar

Ekran	Özellikleri
Giriş (şifre)	İlk açılışta şifre belirleme; uzaktan ilk kurulum engelli.
Görevli paneli	Solda menü, sağda içerik; istatistik kartları, canlı görüntü, olay listesi (3 sn'de bir yenilenir), Test modu, "Bugünü sıfırla".
Öğrenci Kaydı	Form, KVKK onayı, kameradan / dosyadan fotoğraf, düzenleme penceresi.
Sınavlar / Yoklama	Sınav oluşturma formu (arama + tümünü seç), durum rozetleri, geri sayım, canlı yoklama, Excel butonu.
Giriş kiosku	Tam ekran, büyük ✓ ! ✕ işaretleri, ses, göz açıklığı çubuğu ve geri sayımlı canlılık paneli, başlıkta "Giriş açık · kapanmasına 12 dk".
Kayıt kiosku	Aynalı kamera, düz/sağ/sol adım göstergesi, sesli yönlendirme, otomatik çekim.

CSS teknikleri

- **CSS değişkenleri** (custom properties): `:root { --brand: #004684; --vurgu: #0085d0; }` — tüm renkler tek yerden; kurum renkleri (PAÜ laciverti ve mavisi).
- **CSS Grid ve Flexbox:** menü + içerik düzeni, kart ızgaraları.
- **Duyarlı tasarım** (responsive): `@media (max-width: 860px)` ile telefonda menü üste geçer.
- **Erişilebilirlik:** durumlar sadece renkle değil **✓ ! ✕ işaretleriyle** de ayrılır (renk körlüğü). Kurum kimliği gereği yeşil/sarı/kırmızı yerine mavi tonları kullanılır.
- **Satır içi SVG ikonlar:** ikon kütüphanesi indirmeden, vektörel ve her boyutta net.
- Sistem yazı tipi yığını: `system-ui, "Segoe UI", Roboto, "Noto Sans"`.

JavaScript ve tarayıcı API'leri

API / teknik	Projede kullanımı
Fetch API + async/await	Sunucuyla JSON alışverişi: `const veri = await (await fetch("/api/olaylar")).json();`
FormData	Fotoğraf dosyası ve form alanlarını POST ile göndermek.
setInterval / setTimeout	Yoklama (400 ms), saat, mesajın ekranda kalma süresi (4,5 sn; sırada bekleyen varsa 2,5 sn).
Olay kuyruğu	Aynı anda iki öğrenci gelirse mesajlar üst üste binmez, sırayla gösterilir.

Fullscreen API	`document.documentElement.requestFullscreen()` — kiosk tam ekran.
HTML Audio + Blob	Sunucudan gelen WAV: `new Audio(URL.createObjectURL(blob)).play()`.
Web Speech API	Sunucuda Piper yoksa yedek: `speechSynthesis.speak()` ile tarayıcının Türkçe sesi.
<img> ile MJPEG	Canlı görüntü; `onerror`'da 2 sn sonra kendiliğinden yeniden bağlanır.
DOM işlemleri	`textContent` (HTML enjeksiyonuna karşı güvenli), `classList.toggle`.
Optional chaining (`?.`)	`window.speechSynthesis?.cancel()` — özellik yoksa hata vermez.

> **Neden ilk dokunuş gerekli?**
>
> Tarayıcılar, kullanıcı sayfayla etkileşime geçmeden ses çalmaya ve tam ekrana geçmeye izin vermez (autoplay politikası). Bu yüzden kiosk açıldığında bir kez "dokunun" ekranı gösterilir; o dokunuşla hem tam ekran hem ses izni alınır.

> **Kademeli düşüş (graceful degradation)**
>
> Ses için üç kademe: (1) sunucudaki Piper sesi, (2) yoksa tarayıcının kendi sesi, (3) o da yoksa sessiz ama ekranda yazılı. Kamera için: telefon yoksa webcam. Sistem bir parça eksik olsa bile çalışmaya devam eder.

11 Ses: Piper TTS ve ONNX

Piper, Rhasspy projesinin geliştirdiği açık kaynak, **internetsiz** çalışan sinir ağı tabanlı metinden-sese (TTS) motorudur. Projede Türkçe `tr_TR-dfki-medium` sesi kullanılır (~60 MB, `sesler/` klasöründe).

- **Model biçimi — ONNX:** Open Neural Network Exchange, farklı yapay zekâ çatılarında (PyTorch vb.) eğitilmiş modelleri ortak bir biçimde saklar. **ONNX Runtime** bu modeli işlemcide hızlıca çalıştırır. `.onnx.json` dosyası modelin ayarlarını (örnekleme hızı, fonem listesi) içerir.
- **Çıktı:** Python'un `wave` modülüyle bellekte WAV dosyası üretilir, tarayıcıya gönderilir.
- **Önbellek:** Her cümlenin SHA-1 özeti dosya adı olur (`sesler/onbellek/<özet>.wav`). Aynı cümle ("Hoş geldiniz, Ali Yılmaz, Sıra No 12") ikinci kez üretilmez, diskten okunur.
- **Isıtma:** Site açılırken model belleğe yüklenir ve sabit cümleler ("Lütfen gözlerinizi kırpın" vb.) arka planda önceden üretilir; ilk öğrenci 2–3 sn beklemez.
- **Kilit:** Model aynı anda iki cümle üretmesin diye `threading.Lock` kullanılır.

> **Neden bulut tabanlı ses (Google TTS vb.) değil?**
> İnternet bağımlılığı olmaz, öğrenci adları dışarıya gönderilmez (KVKK), gecikme ve ücret yoktur.

12 Excel çıktısı: openpyxl

openpyxl, Excel yüklü olmadan `.xlsx` dosyası okuyup yazan Python kütüphanesidir. Yoklama sayfasındaki "⬇ Excel'e aktar" butonu iki sayfalı bir çalışma kitabı üretir:

- **Yoklama:** sınav adı, tarih ve saatler, "Gelen 28 / 30", her öğrenci için no, ad soyad, durum (Geldi / Geldi (görevli onayı) / Gelmedi), giriş saati, sıra no. Başlıklar kalın ve renkli, durum hücreleri boyalı, sütun genişlikleri ayarlı.
- **Tüm olaylar:** zaman sırasıyla her olay (tekrar deneme, kayıtsız kişi, erken/geç, canlılık yok...).

Dosya diske yazılmaz; bellekte (`io.BytesIO`) oluşturulup doğrudan indirilir (`send_file(..., as_attachment=True)`). Dosya adındaki özel karakterler düzenli ifadeyle temizlenir.

13 Donanım ve işletim sistemi

Kamera seçenekleri

Kaynak	Açıklama
ESP32-CAM (hedef)	Espressif ESP32 mikrodenetleyicisi + OV2640 kamera modülü; Wi-Fi'lı, ~5–10 $ fiyatlı. Görüntüyü ağ üzerinden MJPEG olarak yayınlar (`http://IP:81/stream`). Aynı anda tek izleyiciye görüntü verir. Projenin klasör adı (`esp32`) buradan gelir.
Telefon (şu an)	Android "IP Webcam" uygulaması telefonu ağ kamerasına çevirir (`http://IP:8080/video` , 640×480). Telefon ve bilgisayar aynı ağda olmalı (en sağlamı: bilgisayarı telefonun hotspot'una bağlamak).
Webcam (yedek)	`sahte_kamera.py` bilgisayarın kamerasını ESP32 ile aynı formatta yayınlar. Telefona ulaşılamazsa baslat.py otomatik buna geçer ve masaüstü bildirimi gösterir.

Hepsi aynı arayüzü (HTTP üzerinden MJPEG) kullandığı için kamerayı değiştirmek `ayarlar.py`'de tek satırı değiştirmektir. Yüz tanıma kamerada değil **bilgisayarda** yapılır; ESP32'nin işlemcisi derin sinir ağı için yetersizdir.

Hoparlör: Ses bilgisayardan çıkar; Bluetooth hoparlör bağlayıp varsayılan çıkış yapmak yeterli. **Ekran:** kapıdaki monitör/tablet giriş kioskunu tarayıcıda tam ekran açar.

Linux ve Bash

Sistem Linux (Fedora tabanlı Nobara) üzerinde çalışır. Kullanıcı deneyimi için işletim sisteminin araçları kullanılır:

Araç	Kullanımı
`sinav_giris.sh` (Bash)	Başlat / `durdur` / `kiosk` . Zaten çalışıyorsa sadece tarayıcıyı açar. Site hazır olana kadar `curl` ile bekler (en çok 40 sn).
`sinav-giris.desktop`	freedesktop.org standardı uygulama simgesi; menüde "Sınav Giriş Sistemi", sağ tıkta "Giriş kioskunu aç", "Sistemi durdur".
`nohup … &` , PID dosyası	Sistemi arka planda çalıştırır; süreç kimliği `loglar/sistem.pid`'e yazılır.
`pgrep` , `/proc/PID/cmdline`	Sistemin çalışıp çalışmadığını güvenle anlamak (yanlış süreci öldürmemek).
`kill -TERM`	Düzgün kapatma sinyali; baslat.py yakalayıp tüm servisleri kapatır.
`notify-send`	Masaüstü bildirimi ("Sistem başlatılıyor", "Telefon kamerasına ulaşılamadı").
`xdg-open`	Varsayılan tarayıcıda sayfayı açmak.
`firewall-cmd`	Ağdaki diğer cihazlar için 5000 portunu açmak.
`dnf`	dlib derlemesi için `cmake gcc-c++ python3-devel` kurulumu.
Loglar	Tüm çıktı `loglar/sistem.log`'a; önceki çalıştırma `sistem.onceki.log` .

14 Test: pytest

pytest, Python'un en yaygın test çatısıdır. Projede 8 test dosyasında **124 otomatik test** vardır; hepsi ~15 saniyede çalışır: `venv/bin/python -m pytest`.

Dosya	Neyi doğrular
`test_tanima.py`	Doğru kişi, kayıtsız kişi, "emin değilim", benzerlik %, baş yönü, ayna görüntüsü.
`test_olay_kurallari.py`	Giriş / tekrar / listede yok / kayıtsız / erken / geç, sıra numaraları, salon dolu.
`test_akis.py`	Art arda 3 kare onayı, ayrılma kuralı, kayıt modu, test modu.
`test_canlilik.py`	Göz kırpma ölçümü, fotoğraf saldırısı, kırpmadan ayrılma, sıra bekleme.
`test_kayit.py`	Form ve KVKK kuralları, aynı yüz iki numara, rehberli kayıt, düzenleme.
`test_sinav.py`	Sınav kuralları, tek aktif sınav, otomatik başla/bitir, Demo, elle giriş, Excel.
`test_site.py`	Şifre ve oturum, kiosk mesajları, "Bugünü sıfırla", sayfa JavaScript'inin `node --check` ile söz dizimi kontrolü.
`test_ses.py`	Türkçe ses üretimi ve önbellek.

- **Fixture** (`conftest.py`): her test boş, geçici bir klasörde (`tmp_path`) çalışır; gerçek veritabanına ve fotoğraflara dokunulmaz.
- **Flask test istemcisi:** tarayıcı açmadan sayfalara istek atılır.
- **Sahte saat:** fonksiyonlara `simdi` parametresi verilerek zamana bağlı kurallar (7 sn, 3 sn ayrılma) beklemeden test edilir.
- **Koşullu atlama:** Piper, örnek fotoğraflar ya da Node yoksa ilgili testler `skipif` ile atlanır.

15 Güvenlik ve KVKK

Önlem	Nasıl
Şifre **özeti** (hash)	Şifrenin kendisi değil, Werkzeug'un `generate_password_hash` ile ürettiği tuzlu (salt) özeti saklanır (scrypt). Giriş `check_password_hash` ile doğrulanır.
Kaba kuvvete karşı	Yanlış şifrede 1 sn bekleme.
İlk kurulum yerelden	Şifre ilk kez sadece sistemin çalıştığı bilgisayardan (`127.0.0.1`) belirlenebilir; ağdaki biri "ilk" olamaz.
İmzalı oturum çerezi	Flask oturumu `secrets.token_hex(32)` ile üretilmiş gizli anahtarla imzalanır; çerez değiştirilirse geçersiz olur.
Dosya izinleri	Şifre özeti ve anahtar `0o600` izniyle yazılır.
SQL enjeksiyonu	Tüm sorgular parametreli (`?`).
Girdi doğrulama	Numara sadece rakam (1–20), ad soyad en az iki kelime ve sadece harf (düzenli ifade).

İç servis kapalı	tani.py sadece `127.0.0.1`'de; ağdan erişilemez.
Kimlik bütünlüğü	Aynı yüz iki farklı numarayla kaydedilemez; bir numara başka isimle kullanılamaz.

> **KVKK (6698 sayılı Kişisel Verilerin Korunması Kanunu)**
>
> Yüz görüntüsü **biyometrik veri**, yani **özel nitelikli kişisel veri**dir ve açık rıza gerektirir. Projede: kayıtta açık rıza metni gösterilir ve onay kutusu zorunludur; onay zamanı veritabanına yazılır (`kvkk_onay`); amaç sınırlıdır (sınav girişinde kimlik doğrulama); öğrenci silindiğinde tüm fotoğrafları ve yüz kodları silinir; tüm veriler yerel bilgisayarda kalır, buluta gönderilmez. Ekranda yüzde yalnızca görevli panelinde görünür, kioskta görünmez.

16 Bir öğrencinin girişi: baştan sona

Sunumda sistemi anlatmanın en iyi yolu bir öğrencinin yolculuğunu adım adım anlatmaktır. Her adımda hangi teknolojinin çalıştığı yanında yazılı:

#	Ne oluyor	Teknoloji
1	Kamera saniyede birçok kare yayınlıyor.	ESP32-CAM / IP Webcam, MJPEG, HTTP
2	tani.py'nin kamera iş parçacığı en son kareyi bellekte tutuyor.	OpenCV `VideoCapture`, threading
3	Tanıma iş parçacığı kareyi RGB'ye çeviriyor, yüzü buluyor.	OpenCV, dlib HOG
4	Yüz 128 sayılık koda çevriliyor, kayıtlı kodlarla karşılaştırılıyor: Ali Yılmaz, mesafe 0,38 (%89).	dlib ResNet, NumPy, Öklid mesafesi
5	Art arda 3 analizde aynı kişi → onaylandı.	Python (`AkisDurumu`)
6	Kurallar: aktif sınav var mı, listede mi, giriş penceresi açık mı, daha önce girdi mi? → "giris" olacak.	SQLite sorguları, `datetime`
7	Canlılık bekleniyor; kiosk 400 ms'lik yoklamada bunu görüyor, "Lütfen gözlerinizi kırpın" diyor ve çubuğu gösteriyor.	Flask JSON, fetch, Piper sesi
8	Her karede göz açıklığı ölçülüyor; göz kapanıp açılıyor → kırptı.	Yüz noktaları, EAR, NumPy
9	Boş sıralardan rastgele 12 seçiliyor; olay ve yüz fotoğrafı kaydediliyor.	SQLite `INSERT`, `random`, `cv2.imwrite`
10	Kiosk yeni olayı alıyor: büyük ✓, "Hoş geldiniz, Ali Yılmaz, Sıra No 12".	JavaScript, DOM, CSS
11	Sesli karşılama çalıyor.	Piper + ONNX → WAV → HTML Audio
12	Görevlinin yoklama sayfası 3 sn içinde "29 / 30" oluyor; sınav sonunda Excel indiriliyor.	fetch, openpyxl

Toplam süre: öğrenci kameraya baktıktan sonra yaklaşık 1 saniyede tanınır; göz kırpma ile birlikte giriş 2–3 saniye sürer.

Canlı demo için hatırlatmalar

- Sınav sayfasındaki **Demo** butonu sınavı "10 dk sonra başlıyor" yapıp başlatır: giriş hemen açılır.
- **Test modu** (Tanıma sayfası): açıkken olay yazılmaz, kamera ayarlanabilir.
- Prova sonrası **"Bugünü sıfırla"** veya yoklama sayfasında **"Yoklamayı sıfırla"**.
- Fotoğrafla deneme gösterisi: telefondaki bir yüz fotoğrafını kameraya tutun → tanınır ama göz kırpmadığı için "Canlılık doğrulanamadı".
- Telefon kamerası açılmazsa sistem webcam'le açılır (bildirim gelir) — paniğe gerek yok.

17 Sunumda sorulabilecek sorular

Yapay zekâ modelini siz mi eğittiniz?

Hayır. dlib'in önceden eğitilmiş derin sinir ağını (ResNet) kullanıyoruz; bu "transfer" yaklaşımıdır. Model genel olarak "yüzü sayılara çevirmeyi" öğrenmiştir. Biz her öğrencinin kodunu kaydedip karşılaştırıyoruz; yeni öğrenci için yeniden eğitim gerekmez.

Sistem nasıl "emin" oluyor? Yanlış kişiyi içeri alabilir mi?

Üç güvence var: (1) mesafe 0,47'den küçük olmalı (kütüphane varsayılanı 0,6, biz daha katıyız), (2) en yakın kişi ikinciden en az 0,05 daha yakın olmalı, değilse "?" der, (3) art arda 3 analizde aynı sonuç gerekir. Ayrıca her girişin fotoğrafı saklanır, görevli kontrol edebilir.

Birinin fotoğrafını göstererek girilebilir mi?

Hayır; tanınan kişiden göz kırpması istenir, fotoğraf kırpamaz. Ancak göz kırpan bir videoyla aşılabilir; bu sınırı biliyoruz, ileride rastgele hareket isteme veya derinlik kamerası eklenebilir.

İnternet gerekiyor mu?

Hayır. Yüz tanıma, ses üretimi, veritabanı ve web arayüzü tamamen yerel bilgisayarda çalışır. Kamera ve bilgisayarın aynı yerel ağda olması yeterli.

Neden ESP32 üzerinde tanıma yapmıyorsunuz?

ESP32'nin işlemcisi (240 MHz, ~520 KB RAM) derin sinir ağı için çok zayıf. ESP32 sadece ucuz bir ağ kamerası; ağır iş bilgisayarda.

Neden Flask, neden Django değil?

Flask hafif ve sade; tek bilgisayarda çalışan, ORM ve yönetim paneline ihtiyaç duymayan bu proje için yeterli. Daha az bağımlılık, daha kolay anlaşılır kod.

Neden SQLite?

Sunucu kurulumu gerektirmez, tek dosyadır, Python'la hazır gelir ve bu ölçek için fazlasıyla hızlıdır. Yedeklemek dosyayı kopyalamaktır.

Neden React gibi bir frontend kütüphanesi kullanmadınız?

İnternetsiz çalışma ve sadelik. Sayfalar küçük; saf JavaScript, fetch ve CSS yeterli. Derleme adımı (build) yok.

Aynı anda birden fazla öğrenci gelirse?

Karede birden fazla yüz tanınabilir. Canlılık sırayla istenir (biri bekliyorsa diğeri sırasını bekler) ve kiosk mesajları bir kuyrukta sırayla gösterilir.

İkizler ya da çok benzeyen iki öğrenci?

MARJ kuralı devreye girer: iki kişiye de benzer çıkarsa sistem "?" der ve karar vermez. Görevli "Elle giriş" ile onaylayabilir. `hassasiyet.py` sınıftaki en benzer çiftleri önceden raporlar.

Gözlük, maske, sakal, ışık?

Gözlük ve sakal genelde sorun olmaz (farklı açılardan çoklu fotoğraf yardımcı olur). Maske yüzün yarısını kapattığı için tanımayı zorlaştırır. Işık çok kötüyse HOG yüz bulamayabilir; kapı aydınlık olmalı.

Kişisel veriler nasıl korunuyor?

KVKK açık rıza alınıyor ve zamanı kaydediliyor, veriler yerel bilgisayarda kalıyor, panel şifreli, şifre özet olarak saklanıyor, öğrenci silinince tüm fotoğrafları silinir.

Doğruluğunu nasıl ölçtünüz?

4 kişi, 19 fotoğrafla mesafe ölçümü: aynı kişi 0,26–0,59, farklı kişiler 0,51–0,85. Eşik bu aralıklara göre seçildi. `hassasiyet.py` her fotoğrafı diğerleriyle tanımaya çalışarak doğru / yanlış / emin değil sayılarını verir. Ayrıca 124 otomatik test var.

Ölçeklenebilir mi? 10 salon olsa?

Her salonda bir bilgisayar + kamera ile şu anki haliyle çalışır. Merkezi sistem için veritabanı PostgreSQL'e taşınır, servisler ağ üzerinden konuşur; mimari zaten servislere ayrılmış olduğu için bu geçiş kolaydır.

Ses nasıl üretiliyor?

Piper adlı internetsiz, sinir ağı tabanlı Türkçe TTS ile. Model ONNX biçiminde, ONNX Runtime ile çalışıyor; üretilen cümleler önbelleğe alınıyor.

18 Sözlük

Terim	Anlamı
API	Programların birbirleriyle konuştuğu arayüz; burada JSON döndüren web adresleri.
Backend / Frontend	Sunucu tarafı (Python, Flask, veritabanı) / kullanıcının tarayıcıda gördüğü taraf (HTML, CSS, JS).
Biyometrik veri	Yüz, parmak izi gibi kişiyi tekil olarak tanımlayan fiziksel özellik verisi.
Canlılık tespiti (liveness)	Kameradaki yüzün gerçek, canlı bir insan olduğunu doğrulama.
Çerez (cookie)	Tarayıcının sitede sakladığı küçük veri; oturum bilgisi.
Derin sinir ağı	Çok katmanlı yapay sinir ağı; örneklerden öğrenen model.
EAR	Eye Aspect Ratio, göz açıklık oranı (yükseklik / genişlik).
Embedding / yüz kodu	Yüzü temsil eden 128 sayılık vektör.
Endpoint / rota	Sunucudaki belirli bir adres ve onu karşılayan fonksiyon.
Framework (çatı)	Uygulama yazmak için hazır iskelet (Flask).
Hash (özet)	Veriden tek yönlü üretilen sabit uzunlukta değer; geri çevrilemez.
HOG	Yönlü gradyan histogramı; klasik yüz bulma yöntemi.
İş parçacığı (thread)	Aynı program içinde eşzamanlı çalışan iş kolu.
JSON	Metin tabanlı veri biçimi: `{"isim": "Ali", "sira": 12}`.
Kiosk	Halka açık, tek amaçlı, tam ekran çalışan ekran.
Landmark	Yüzdeki önemli noktalar (göz köşeleri, burun ucu...).
MJPEG	Videoyu art arda JPEG resimleri olarak taşıyan biçim.
ONNX	Yapay zekâ modelleri için ortak dosya biçimi.
Polling (yoklama)	Sunucuya düzenli aralıklarla "yeni bir şey var mı?" diye sorma.
Port	Bir bilgisayarda bir servisin dinlediği numara (5000, 8082).
Proxy (vekil)	İsteği başka servise aktaran ara katman (site.py → tani.py görüntüsü).
ResNet	"Artık bağlantılı" derin sinir ağı mimarisi.
SQL enjeksiyonu	Girdiye SQL kodu sokarak veritabanını ele geçirme saldırısı.
TTS	Text-to-Speech, metinden sese.
Vanilla JS	Hiçbir kütüphane kullanılmadan yazılmış saf JavaScript.
venv	Python'un projeye özel sanal ortamı.